参考资料与样本索引

这份附录是 Skill 工程实战指南 的配套速查。当你在实践中需要快速找到某个工具、某个样本、或者某个判断的出处时，来这里翻。

Part 1：推荐样本速查

生态入口

名称	类型	最适合解决什么	注意不要误读成什么
skills.sh	目录站	发现现成 skill、扩搜视野	质量背书系统
awesome-copilot	社区聚合	看教程、扩样本、了解社区趋势	工程基座
vercel-labs/agent-skills	样板库	学 skill 结构、学 Use when、学分层	installer 或治理平台
vercel-labs/skills	installer	受控安装、更新、定位到 project/global scope	信任或效果验证系统
skill-forge	治理层	补审计、发布流程、质量门槛	冷启动阶段的唯一模板

值得拆读的进阶样本

agent-skills（学生命周期组织）：看它怎么把 Define → Plan → Build → Verify → Review → Ship 串成一组连贯的 skill。重点不是命令数量，而是阶段切分和验证纪律。

- 本地拆解路径： `/Users/bowhead/ai_dev_skill/addyosmani-agent-skills-analysis/eval_skills/`

get-shit-done（学系统分层）：看 command / agent / workflow / reference / template 五层结构。它说明 skill engineering 不只能长成标准 SKILL.md 卡片，也能演化成 runtime 系统。

- 本地拆解路径： `/Users/bowhead/ai_dev_skill/get-shit-done-analysis/eval_skills/`

superpowers（学注入和验证）：看 bootstrap、hooks、宿主适配、行为测试。它说明 skill 可以从"内容仓库"进化成 workflow-enforcing plugin。

- 本地拆解路径：`/Users/bowhead/ai_dev_skill/superpowers-analysis/eval_skills/`

gstack（看高级系统）：看模板治理、浏览器 runtime、specialist review、多宿主适配。适合已经写过几个 skill、想看更成熟系统的人。

- 本地拆解路径：`/Users/bowhead/ai_dev_skill/gstack-analysis/eval_skills/`

拆读任何样本时的 6 个问题

1. 它到底在解决一个任务、一个流程阶段、一个角色，还是一个完整系统?
2. 它的入口层是什么？（description 写了什么？路由够精准吗？）
3. 它的正文和 supporting layer 是怎么分的?
4. 它最值得借的是结构、路由、边界，还是编排?
5. 它最容易被照抄的是哪一层？（通常就是最不该照抄的）
6. 如果装进真实项目，最先需要补的安全措施是什么?

Part 2：工具角色速查

推荐的组合式 baseline

当前最稳的方案不是找一个"总冠军"，而是按职责组合：

层	推荐对象	它负责什么
样板库	vercel-labs/agent-skills	提供高质量结构参考和内容样本
安装层	vercel-labs/skills	受控安装、更新、查找、兼容
治理层	skill-forge	审计、质量门槛、发布流程
学习入口	skills.sh / awesome-copilot	发现、学习、扩搜

按阶段的采用建议

冷启动学习：先看 skills.sh → awesome-copilot → agent-skills，目标是长眼睛、学结构。

建立最小 workflow：先搭 agent-skills（学） + skills（装） + 自己的 trust gate，目标是能写、能装、能验证。

团队化治理：再把 skill-forge、版本纪律、A/B 评测和 role-based bundles 补上。

常见误判速查

误判	更稳的理解
在 skills.sh 上能找到 → 应该靠谱	目录站解决发现，不是质量保证
vercel-labs/skills 能装 → 装进去就行	installer 解决装载，不是效果验证
agent-skills 最完整 → 就用它当基座	样板库强在教你学，不等于全链路可押
被很多地方收录 → 可以直接用	被收录说明可发现性高，不说明质量高

一句话：**能找到 ≠ 能信，能安装 ≠ 有效，很好学 ≠ 全链路能押。**

Part 3：研究证据索引

> 这部分面向需要向团队论证"为什么这样做"的读者。如果你只是自己用，可以跳过。

本指南的核心判断来自一轮系统性研究，拆分为三个主题：

- **Topic 01**：方法论与规范接口——skill 的最小共同层、portable core、progressive disclosure
- **Topic 02**：工具链与生命周期——样板、安装、治理、发布、评测的职责拆分
- **Topic 03**：生态信号与采用判断——哪些入口值得学、哪些信号值得信、哪些不值得

研究原始材料位于 `../topics/` 目录下，包括：

文件	内容
`01-skill-methodology-and-spec.md`	方法论与规范的深度研究
`02-skill-toolchain-and-lifecycle.md`	工具链与生命周期的深度研究
`03-ecosystem-signals-and-adoption.md`	生态信号与采用判断的深度研究
`_artifacts/W2-final-recommendation-and-baseline.md`	最终推荐与 baseline 方案
`_artifacts/W2-combination-baseline-workflow-draft.md`	组合式 workflow 草案

文件	内容
`_artifacts/W2-formal-comparison-table.md`	正式比较表
`_reference/`	各对象的详细 teardown 和证据文件

主要证据映射：

指南中的判断	主要证据来源
"先读后编"路径更快	Topic 03 学习杠杆分析 + 官方 guide
不要追单一赢家，要做职责拆分	跨 Topic 综合 + 正式比较表
样板库值得先读	agent-skills 本地拆解 + 官方样板分析
installer ≠ trust system	vercel-labs/skills CLI 分析
发现容易 ≠ 可信	skills.sh / awesome-copilot 分析 + 独立效果基准
先写 portable core	方法论收敛分析 + 跨平台边界研究
role-based bundles 防止 selection failure	组合式 workflow 草案